

**DAY 3**

# **Git Workflow & Branches**

**GitFlow**

**Trunk-Based Dev**

**Protected Branches**

**CI/CD Triggers**

|                                                                |                                     |
|----------------------------------------------------------------|-------------------------------------|
| Table of Contents .....                                        | <b>Error! Bookmark not defined.</b> |
| 1. Why Branching Is the Foundation of CI/CD .....              | 3                                   |
| What Goes Wrong Without a Strategy .....                       | 3                                   |
| 2. GitFlow .....                                               | 4                                   |
| 2.1 The Problem GitFlow Was Built to Solve .....               | 4                                   |
| 2.2 The Five Branch Types.....                                 | 4                                   |
| 2.3 The GitFlow Workflow, Step by Step .....                   | 4                                   |
| 2.4 release/* vs hotfix/* — Understanding the Difference ..... | 5                                   |
| 2.5 When to Use GitFlow.....                                   | 5                                   |
| 3. Trunk-Based Development .....                               | 6                                   |
| 3.1 The Core Idea .....                                        | <b>Error! Bookmark not defined.</b> |
| 3.2 Feature Flags — The Enabler of TBD.....                    | <b>Error! Bookmark not defined.</b> |
| 3.3 GitFlow vs Trunk-Based — Full Comparison .....             | 9                                   |
| 4. Protected Branches .....                                    | 10                                  |
| 4.1 Why Branch Protection Matters .....                        | 10                                  |
| 4.2 What You Can Configure in GitLab.....                      | 10                                  |
| 4.3 What the Full Flow Looks Like With Protection .....        | <b>Error! Bookmark not defined.</b> |
| 5. Branch Naming and CI/CD Triggers.....                       | 11                                  |
| 5.1 How GitLab Knows Which Branch Triggered the Pipeline.....  | 11                                  |
| 5.2 Standard Branch Naming Convention .....                    | 11                                  |
| 5.3 Writing Pipeline Rules — Three Patterns.....               | 12                                  |
| Pattern 1: Exact Match.....                                    | 12                                  |
| Pattern 2: Regex Match .....                                   | 12                                  |
| Pattern 3: Combining Rules for a Full Pipeline .....           | 13                                  |
| 5.4 How the Three Concepts Connect.....                        | 14                                  |
| 6. Common Mistakes to Avoid.....                               | 15                                  |
| 7. Quick Reference .....                                       | 16                                  |
| 7.1 Branch Naming Cheatsheet.....                              | 16                                  |
| 7.2 GitLab CI Rules Cheatsheet .....                           | 16                                  |
| 7.3 Strategy Selection Guide .....                             | 17                                  |
| 8. Hands-On Lab .....                                          | 18                                  |
| 8.1 Lab Steps .....                                            | 18                                  |
| 8.2 Expected Results .....                                     | 19                                  |
| 8.3 Bonus Task .....                                           | 19                                  |
| 8.4 Review Questions .....                                     | 19                                  |
| 9. Summary .....                                               | 20                                  |

# 1. Why Branching Is the Foundation of CI/CD

---

Before writing a single line of pipeline code, you need to understand one fundamental idea: a GitLab pipeline does not run on its own schedule. Every pipeline is triggered by something you do in Git — and in most cases, that trigger is determined by the branch you pushed to.

This means your branching strategy and your CI/CD design are the same decision. The branch names you choose, how long branches live, and who can push where — all of these directly control how your pipelines behave.

## Remember

Your branching strategy decides three things about every pipeline:

1. WHEN it runs — which Git events trigger execution
2. WHAT it runs — which stages and jobs are included
3. WHETHER code gets deployed — which branches have deploy authority

## What Goes Wrong Without a Strategy

Without a defined branching model, teams hit the same problems repeatedly:

- Every push triggers a deployment, including half-finished feature code
- No one knows which branch is safe to branch from
- Pipeline rules are added ad-hoc and start contradicting each other
- Merge conflicts pile up because branches live for too long
- Production breaks because there was no gate between "written" and "deployed"

## Watch Out

A common beginner mistake: designing the pipeline first and adding branching rules later. The branching strategy must come first. The pipeline is built around it, not the other way.

There are two dominant branching strategies in the industry today: GitFlow and Trunk-Based Development. Each has a different philosophy, different complexity, and different CI/CD implications. You need to understand both.

## 2. GitFlow

### 2.1 The Problem GitFlow Was Built to Solve

Imagine a team of ten developers all committing to a single main branch. On day one, everything works. By day five, one developer pushed a half-finished feature that broke the build. Another deployed without testing. Now the entire team is blocked because no one can trust what is in main.

GitFlow, introduced by Vincent Driessen, was the answer to this chaos. The idea was simple: give every type of work its own dedicated branch with its own rules. Main becomes sacred — only stable, fully tested, released code ever lives there.

### 2.2 The Five Branch Types

GitFlow uses five branch types, each with a specific purpose:

| Branch    | Lifetime       | Created From | Merges Into    | Deploy? | Purpose                                                                                             |
|-----------|----------------|--------------|----------------|---------|-----------------------------------------------------------------------------------------------------|
| main      | Permanent      | —            | —              | ☑ YES   | Production code only. Deployment always happens from here.                                          |
| develop   | Permanent      | main (once)  | —              | ✗ NO    | Integration branch. The next release being assembled.                                               |
| feature/* | Days to weeks  | develop      | develop        | ✗ NO    | One feature per branch. Isolated development.                                                       |
| release/* | Days           | develop      | main + develop | ✗ NO    | Feature freeze. QA and bugfixes only. Merges into main — deploy happens after that merge, not here. |
| hotfix/*  | Hours to 1 day | main         | main + develop | ✗ NO    | Emergency fix. Merges into main — deploy happens after that merge.                                  |

### 2.3 The GitFlow Workflow, Step by Step

Understanding how code flows through GitFlow is more important than memorising the branch names. Here is a complete sprint cycle:

1. A developer creates feature/add-payment from the develop branch.
2. They work on the feature, committing regularly. The pipeline runs tests on each push.
3. When ready, they open a Merge Request into develop. After review and pipeline pass, it merges.
4. At the end of the sprint, the release manager creates release/v1.2 from develop.
5. QA tests on release/v1.2. Two bugs are found and fixed directly on this branch. No new features are added.
6. The release is approved. Merge release/v1.2 into main and tag it v1.2.0.

- 7. Also merge release/v1.2 back into develop so the bugfixes are not lost in future development.
- 8. Three days later, a critical bug is found in production. Create hotfix/db-fix from main.
- 9. Fix committed. Merge hotfix into main (tag it v1.2.1) AND back into develop.

 **Watch Out**

The most common GitFlow mistake: forgetting to merge release/\* and hotfix/\* back into develop. If a hotfix only merges into main, develop does not have the fix. It will reappear in the next release.

Both release/\* and hotfix/\* must always merge into main AND develop.

2.4 release/\* vs hotfix/\* — Understanding the Difference

These two branch types are often confused because they both eventually merge into main. The key distinction is where they come from and why they exist:

|              | release/*                        | hotfix/*                   |
|--------------|----------------------------------|----------------------------|
| Origin       | Branched from develop            | Branched from main         |
| Trigger      | Planned release cycle            | Emergency production issue |
| Work allowed | Bug fixes only (no new features) | Minimal fix only           |
| Timeline     | Days (QA cycle)                  | Hours (urgent)             |
| Example      | release/v2.0.0                   | hotfix/security-cve-2024   |

2.5 When to Use GitFlow

GitFlow is a strong choice in specific contexts, but it is the wrong tool for others:

| GitFlow Works Well For                           | GitFlow Is a Poor Fit For                    |
|--------------------------------------------------|----------------------------------------------|
| Versioned software (v1.0, v2.0, v3.0)            | Web apps or SaaS deployed daily              |
| Scheduled releases (weekly, monthly)             | Teams deploying multiple times per day       |
| Large teams with strict change control           | Small, fast-moving teams                     |
| Supporting multiple live versions simultaneously | Microservices with independent deploy cycles |
| Regulated industries with audit requirements     | Startups prioritising speed over control     |

 **Remember**

GitFlow is less preferred in modern DevOps because CI/CD pipelines work best with frequent integration. The multi-branch promotion process — feature → develop → release → main — slows down delivery and makes pipeline rules significantly more complex.

## 3. Trunk-Based Development

### Trunk-Based Development (TBD)

TBD says:

“Do not stay away from main for too long.”

Instead:

- Create very small branches
- Finish small work quickly
- Merge into main daily

So instead of:

feature/dashboard

for 2 weeks...

You split work into tiny parts:

feature/dashboard-layout

feature/dashboard-api

feature/dashboard-connect

Each branch lives:

- 1 day
- maybe 2 days max

Then merged immediately.

---

### But Here Is the Big Question

You asked correctly:

“If feature is incomplete, how can we merge to main?”

This is where **Feature Flags** come in.

---

### Feature Flag = ON/OFF Switch

Imagine your application has a hidden switch.

Like:

if flag\_on:

    show\_new\_feature()

else:

    show\_old\_feature()

---

### Real-Life Example

Suppose Netflix is building a new homepage.

Developers finish only 20%.

Normally:

- They cannot deploy incomplete code

But with feature flags:

```
if feature_flags.is_enabled("new_homepage"):
    return new_homepage()
else:
    return old_homepage()
```

Now:

| Flag Status | User Sees    |
|-------------|--------------|
| OFF         | Old homepage |
| ON          | New homepage |

So developers can:

- merge unfinished code
- deploy safely
- users still see old feature

---

## Why This Is Powerful

Because now:

### Deployment ≠ Release

These are different things.

---

### Deployment

Code goes to production server.

Example:

kubectl apply

docker push

Code exists in production.

---

### Release

Users actually see the feature.

Controlled by:

- feature flag ON/OFF
- 

### Visual Flow

Developer writes code



Merge to main quickly



Deploy to production



Feature flag OFF

↓  
Users still see old behaviour

Later:

Turn feature flag ON

↓  
Users now see new feature

NO redeployment needed.

---

### Instant Rollback

Suppose new feature breaks production.

Without flags:

- rollback deployment
- redeploy old version

With flags:

- simply turn OFF feature

Done in seconds.

---

### Simple Analogy

Imagine constructing a mall.

GitFlow approach:

- Entire mall hidden for 2 years
- Open everything together

TBD approach:

- Build floor by floor
- Keep unfinished shops covered
- Customers only see completed areas

Feature flags are those “UNDER CONSTRUCTION” curtains.

---



### 3.3 GitFlow vs Trunk-Based — Full Comparison

| Aspect                        | GitFlow                        | Trunk-Based Development          |
|-------------------------------|--------------------------------|----------------------------------|
| Number of long-lived branches | 2 (main + develop)             | 1 (main only)                    |
| Feature branch lifetime       | Days to months                 | Hours to 2 days max              |
| How often code reaches main   | Per sprint or release          | Multiple times per day           |
| Handling unfinished features  | Lives in long-lived branch     | Merged behind feature flag       |
| Merge conflict frequency      | High                           | Very low                         |
| CI/CD pipeline complexity     | High — 5+ branch type rules    | Low — 2 branch types             |
| Deployment frequency          | Weekly or monthly              | Multiple times daily             |
| Rollback approach             | Complex multi-branch operation | Revert last commit or flip flag  |
| Best suited for               | Enterprise, scheduled releases | SaaS, web apps, microservices    |
| Used by                       | Traditional enterprise teams   | Google, Netflix, Spotify, Amazon |

## 4. Protected Branches

### 4.1 Why Branch Protection Matters

Without branch protection, any developer with repository access can push directly to main at any time. This means untested, unreviewed code can reach production instantly. It happens more often than you would expect — not through malice, but through a simple mistake: a developer types

Without branch protection, any developer can type `git push origin main` and push untested, unreviewed code directly to production. This is not theoretical — it is one of the most common ways production outages start.

Protected branches solve this by enforcing rules at the GitLab server level. No matter what the developer does locally, GitLab rejects pushes that violate the protection rules. The only path to getting code into main is through a Merge Request that has passed all the gates.

### 4.2 What You Can Configure in GitLab

Navigate to your project in GitLab: Settings → Repository → Protected Branches. The following settings are available:

| Setting                  | Recommended Value | What It Enforces                              |
|--------------------------|-------------------|-----------------------------------------------|
| Allowed to push          | No one            | All changes must come through a Merge Request |
| Allowed to merge         | Maintainers only  | Senior review required before main changes    |
| Required approvals       | Minimum 2         | Four-eyes principle — two people must review  |
| Pipelines must succeed   | Enabled           | No broken code can reach main                 |
| All discussions resolved | Enabled           | Every review comment must be addressed        |
| Force push               | Blocked for all   | Commit history cannot be rewritten            |

#### Remember

Force push is blocked for an important reason: it rewrites Git history. If history is rewritten on main, the audit trail of what changed and when is destroyed. In regulated industries (banking, healthcare, government), this audit trail is a legal requirement.

## 5. Branch Naming and CI/CD Triggers

### 5.1 How GitLab Knows Which Branch Triggered the Pipeline

When any pipeline runs in GitLab, the platform automatically sets a variable called

When a pipeline runs, GitLab automatically sets a variable called `$CI_COMMIT_BRANCH`. You do not create or configure this variable — GitLab sets it for you. It contains the exact name of the branch that triggered the pipeline.

```
# GitLab sets this automatically on every pipeline run
# You can reference it in any job condition

$CI_COMMIT_BRANCH = "main"
$CI_COMMIT_BRANCH = "feature/add-login"
$CI_COMMIT_BRANCH = "bugfix/payment-error"
$CI_COMMIT_BRANCH = "release/v2.1.0"
$CI_COMMIT_BRANCH = "hotfix/security-patch"
```

Your pipeline YAML uses this variable to decide which jobs run on which branches. This is why branch naming is not just an organisational preference — it directly controls your pipeline behaviour.

### 5.2 Standard Branch Naming Convention

GitLab CI rules use pattern matching against branch names. If your team uses inconsistent names, your rules will not match and your pipeline will behave unpredictably. Always follow this convention:

```
# Pattern: <type>/<short-description>    (lowercase, hyphens, no spaces)

feature/add-user-login
feature/payment-gateway-integration
bugfix/null-pointer-in-checkout
bugfix/payment-timeout
hotfix/security-patch-cve-2024
release/v2.1.0
chore/update-dependencies
```

#### ⚠ Watch Out

Branch names are case-sensitive in CI rules. These will NOT match `feature/*` rules:

- ✗ Feature/Add-Login (capital F)
- ✗ feature\_add\_login (underscores instead of slash + hyphens)
- ✗ featureAddLogin (camelCase, no separator)
- ✓ feature/add-login (correct: lowercase, slash, hyphens)

## 5.3 Writing Pipeline Rules — Three Patterns

### Pattern 1: Exact Match

Use `==` for exact branch name comparison. The most common use: run deploy only on main.

```
deploy-job:
  stage: deploy
  script:
    - echo "Deploying to production"
  rules:
    - if: $CI_COMMIT_BRANCH == "main" # case-sensitive exact match
```

| Branch pushed    | deploy-job runs?     |
|------------------|----------------------|
| main             | ☑ YES — exact match  |
| Main (capital M) | ✗ NO — case mismatch |
| develop          | ✗ NO — not 'main'    |
| feature/login    | ✗ NO — not 'main'    |

### Pattern 2: Regex Match

Use `==~` with a regex pattern to match a family of branches. Essential for matching all `feature/*` or `bugfix/*` branches.

```
test-job:
  stage: test
  script:
    - npm test
  rules:
    - if: $CI_COMMIT_BRANCH ==~ /^feature\/.*/

# Regex breakdown:
# ==~           → "matches" operator
# / /          → regex written between these slashes
# ^            → must START with what follows
# feature\/    → the literal text "feature/" (backslash escapes the slash)
# .*           → followed by any characters (any length)
```

| Branch name       | Matches? | Why                                    |
|-------------------|----------|----------------------------------------|
| feature/add-login | ☑ YES    | Starts with 'feature/'                 |
| feature/payment   | ☑ YES    | Starts with 'feature/'                 |
| feature           | ✗ NO     | No slash — pattern requires 'feature/' |
| my-feature/login  | ✗ NO     | Does not start with 'feature/'         |
| bugfix/error      | ✗ NO     | Starts with 'bugfix/', not 'feature/'  |

### Pattern 3: Combining Rules for a Full Pipeline

In a real project, different jobs run on different branches. This is the standard pattern used in professional CI/CD setups:

```
stages:
  - test
  - build
  - deploy

# — Runs on every branch (all developers get test feedback) —
unit-tests:
  stage: test
  script: [ "npm test" ]
  rules:
    - if: $CI_COMMIT_BRANCH

# — Runs only on feature branches —
dependency-scan:
  stage: test
  script: [ "trivy fs ." ]
  rules:
    - if: $CI_COMMIT_BRANCH =~ /^feature\/.*/

# — Runs only when code reaches main —
build-image:
  stage: build
  script: [ "docker build -t myapp:$CI_COMMIT_SHA ." ]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# — Deploy to production — requires human approval —
deploy-production:
  stage: deploy
  script: [ "kubectl set image deployment/myapp myapp=myapp:$CI_COMMIT_SHA" ]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual # pipeline pauses here; someone must click GO
```

| Job               | feature/add-login | bugfix/fix-error | main             |
|-------------------|-------------------|------------------|------------------|
| unit-tests        | ☑ Runs            | ☑ Runs           | ☑ Runs           |
| dependency-scan   | ☑ Runs            | ✗ Skipped        | ✗ Skipped        |
| build-image       | ✗ Skipped         | ✗ Skipped        | ☑ Runs           |
| deploy-production | ✗ Skipped         | ✗ Skipped        | ☑ Manual trigger |

 **Remember**

deploy-production uses when: manual even on main. This means the pipeline pauses at that job and waits for someone to click the play button in GitLab. Automated tests pass without human involvement. The decision to deploy to production

always involves a human. This is the standard approach in most companies.

## 5.4 How the Three Concepts Connect

Branch naming, protected branches, and pipeline rules work together as a single system. Here is the complete flow of a code change in a well-configured repository:

```
1. Developer creates:  git checkout -b feature/add-payment
                       → Follows naming convention: feature/* rules will match

2. Developer pushes:   git push origin feature/add-payment
                       → Direct push to main is blocked by protection rules
                       → Pipeline triggers: unit-tests + dependency-scan run
                       → Developer gets fast feedback

3. Developer opens Merge Request into main
                       → Pipeline runs again on the MR
                       → Must pass before merge is allowed (protection rule)

4. Two reviewers approve. All comments resolved.
                       → Protection rule satisfied: merge is now possible

5. Maintainer merges into main
                       → Protection rule: only maintainers can merge
                       → Main pipeline triggers: build-image + deploy-
production (manual)

6. Maintainer clicks play on deploy-production
                       → Application deployed to production
                       → Full audit trail: MR, approvals, pipeline results all
recorded
```

### ✓ Best Practice

This flow — naming convention + protection rules + branch-based pipeline rules — is how production CI/CD systems work at companies like Amazon, Netflix, and Google. Master this pattern and you understand the core of professional DevOps workflow.

## 6. Common Mistakes to Avoid

| Mistake                                                                   | Problem It Causes                                                                        | The Fix                                                                                                              |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| No branch naming convention                                               | CI rules using <code> =~ /feature\./</code> never match. Pipelines behave unpredictably. | Define and enforce: <code>feature/*</code> , <code>bugfix/*</code> , <code>hotfix/*</code> , <code>release/*</code>  |
| Capital letters in branch names                                           | Regex and <code>==</code> comparisons are case-sensitive. Rules silently fail.           | Always use lowercase. <code>feature/add-login</code> , not <code>Feature/Add-Login</code>                            |
| No protection on main                                                     | Anyone can push directly to production at any time.                                      | Settings → Repository → Protected Branches → protect main immediately                                                |
| Deploy job runs on all branches                                           | Feature branch code gets deployed to production accidentally.                            | Add: rules: - if: <code>\$CI_COMMIT_BRANCH == 'main'</code> to all deploy jobs                                       |
| Forgetting to merge <code>release/*</code> back into <code>develop</code> | Bugfixes made during release prep are lost in future development.                        | Always merge both ways: <code>release/*</code> → <code>main</code> AND <code>release/*</code> → <code>develop</code> |
| Using GitFlow for a daily-deploy SaaS app                                 | Slow delivery, complex pipelines, unnecessary ceremony.                                  | Use Trunk-Based Development for web apps and microservices                                                           |
| Branch lives longer than 2 days (TBD teams)                               | Drift accumulates. Merge conflict becomes expensive.                                     | Break work into smaller pieces. Use feature flags for unfinished work                                                |

## 7. Quick Reference

### 7.1 Branch Naming Cheatsheet

| Type    | Pattern           | Example             | Pipeline Behaviour          |
|---------|-------------------|---------------------|-----------------------------|
| Feature | feature/<name>    | feature/add-login   | Lint, unit tests only       |
| Bug fix | bugfix/<issue>    | bugfix/null-pointer | Lint, unit tests only       |
| Hot fix | hotfix/<fix>      | hotfix/sec-patch    | Lint, tests, fast deploy    |
| Release | release/<version> | release/v2.1.0      | Full pipeline + staging     |
| Main    | main              | main                | Full pipeline + prod deploy |

### 7.2 GitLab CI Rules Cheatsheet

```
# Run on ALL branches
rules:
  - if: $CI_COMMIT_BRANCH

# Run ONLY on main
rules:
  - if: $CI_COMMIT_BRANCH == "main"

# Run ONLY on feature branches
rules:
  - if: $CI_COMMIT_BRANCH =~ /^feature\/.*/

# Run on main OR feature branches
rules:
  - if: $CI_COMMIT_BRANCH == "main"
  - if: $CI_COMMIT_BRANCH =~ /^feature\/.*/

# Run on BOTH feature/* AND bugfix/* (OR inside regex)
rules:
  - if: $CI_COMMIT_BRANCH =~ /^(feature|bugfix)\/.*/

# Run on main automatically, run manually on other branches
rules:
  - if: $CI_COMMIT_BRANCH == "main"
    when: on_success
  - when: manual

# Run on Merge Requests only
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```



## 7.3 Strategy Selection Guide

| Your situation                                                   | Recommended strategy    |
|------------------------------------------------------------------|-------------------------|
| Building versioned software (v1.0, v2.0) with monthly releases   | GitFlow                 |
| Large enterprise team, strict change control, audit requirements | GitFlow                 |
| Building a web app or SaaS, deploying daily or more often        | Trunk-Based Development |
| Small or medium team, fast iteration, CI/CD is a priority        | Trunk-Based Development |
| Microservices with independent deployment cycles                 | Trunk-Based Development |
| Supporting multiple live versions simultaneously                 | GitFlow                 |

## 8. Hands-On Lab

---

This lab puts everything together. You will set up a GitLab repository with proper branch naming, a protected main branch, and a pipeline that behaves differently based on which branch is pushed to. Complete every step on your own GitLab account.

### 8.1 Lab Steps

1

**Create a new GitLab project**

Name it gitlab-branching-lab. Initialise with a README. Set visibility to private.

2

**Protect the main branch**

Go to Settings → Repository → Protected Branches. Add main with: no direct push for anyone, 1 approval required, pipelines must succeed before merge.

3

**Create a feature branch with correct naming**

Run: `git checkout -b feature/add-hello-script`. This follows the convention. The CI rules will match it.

4

**Add a .gitlab-ci.yml with branch-based rules**

Copy the full pipeline from Section 5.3 Pattern 3 into a .gitlab-ci.yml file. Commit and push to the feature branch.

5

**Observe which jobs run on the feature branch**

Go to CI/CD → Pipelines. Confirm that only unit-tests and dependency-scan ran. build-image and deploy-production should be skipped.

6

**Open a Merge Request into main**

Create an MR from feature/add-hello-script into main. Observe the pipeline running again on the MR.

7

**Try to push directly to main**

Run: `git push origin HEAD:main`. GitLab should reject this with a permission error. This confirms the branch protection is working.

8

**Create a release branch and observe pipeline differences**

Run: `git checkout -b release/v1.0`. Push it. Compare which jobs run vs the feature branch. release/v1.0 does not match `/^featureV.*/` so dependency-scan should be skipped.

## 8.2 Expected Results

| Branch                   | unit-tests | dependency-scan | build-image | deploy-production |
|--------------------------|------------|-----------------|-------------|-------------------|
| feature/add-hello-script | ☑ Runs     | ☑ Runs          | ✗ Skipped   | ✗ Skipped         |
| release/v1.0             | ☑ Runs     | ✗ Skipped       | ✗ Skipped   | ✗ Skipped         |
| main (after merge)       | ☑ Runs     | ✗ Skipped       | ☑ Runs      | ☑ Manual          |

## 8.3 Bonus Task

Add a new job called integration-tests that runs only on release/\* branches. Push to your release/v1.0 branch and confirm it runs:

```
integration-tests:
  stage: test
  script:
    - echo "Running full integration test suite"
  rules:
    - if: $CI_COMMIT_BRANCH =~ /^release\/.*/
```

## 8.4 Review Questions

Use these questions to check your understanding after completing the lab:

10. What error did GitLab return when you tried to push directly to main?
11. How many jobs ran when you pushed to the feature branch, and which were skipped?
12. What would happen if the branch was named Feature/Add-Hello with a capital F?
13. Why does deploy-production require a manual trigger even though it is on main?
14. What regex pattern would match both feature/\* AND bugfix/\* branches in one rule?

### 💡 Example

Answer for Q5:

```
if: $CI_COMMIT_BRANCH =~ /^(feature|bugfix)\.*/
```

The | character means OR inside a regex group. This is a pattern you will use frequently.

## 9. Summary

---

The four ideas to take away from this session:

| Core Idea                                        | Why It Matters                                                                                 |
|--------------------------------------------------|------------------------------------------------------------------------------------------------|
| Branching strategy IS CI/CD design               | You cannot write correct pipeline rules without deciding your branch model first.              |
| Trunk-Based is the modern DevOps default         | Simpler pipelines, faster releases, fewer merge conflicts. Used by the largest tech companies. |
| Protected branches prevent production disasters  | Every team needs this. Gate production changes behind review, pipeline success, and approvals. |
| Branch names directly control pipeline behaviour | Consistent naming = predictable automation. Inconsistent naming = silent pipeline failures.    |

**DevOps Shack**

*Build Real. Ship Fast. Learn Deep.*

devopsshack.com | @devopsshack | Aditya Jaiswal